

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



Image Segmentation

Today's Agenda

Time	Topic
13:00 - 14:15	Theory: Image Segmentation & U-Net
14:15 - 14:30	 Break
14:30 - 16:00	Lab: Medical Image Segmentation

Learning Objectives

By the end of this session, you will:

1. **Understand** the three types of segmentation (semantic, instance, panoptic)
2. **Explain** the U-Net encoder-decoder architecture and skip connections
3. **Apply** segmentation loss functions (Dice, Focal, BCE)
4. **Build** a medical image segmentation model in PyTorch

Pre-Class Reading

Did you read the U-Net paper?

Key concepts to have seen:

- Encoder-decoder structure
- Skip connections (different from ResNet residual connections!)
- Transposed convolutions / upsampling

Paper: Ronneberger et al. (2015):

"U-Net: Convolutional Networks for Biomedical Image Segmentation"

1. From Detection to Segmentation

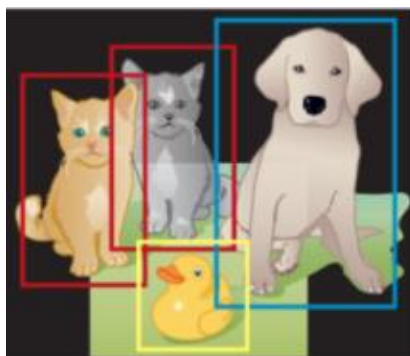
The Final Step in Visual Understanding

The Progression of Visual Understanding

Each step adds more
spatial understanding



Task	Output	Example
Classification	Single label	"Cat"
Localization	Label + 1 bounding box	"Cat" at (x, y, w, h)
Object Detection	Multiple labels + boxes	"Cat" at (...), "Dog" at (...)
Segmentation	Pixel-level labels	Every pixel classified



Cat, Cat, Duck, Dog



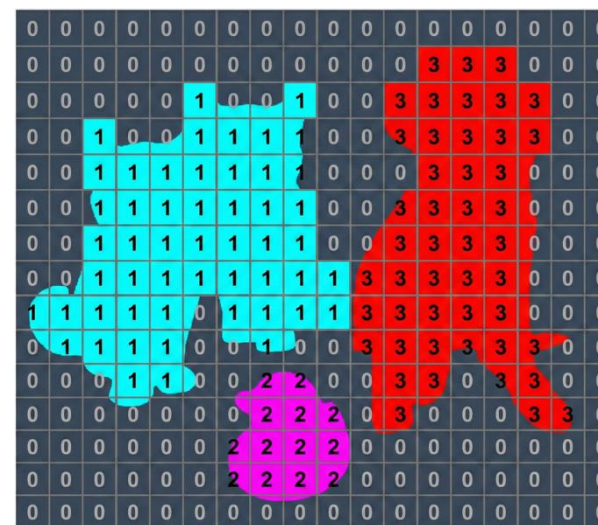
■ Cat | ■ Dog | ■ Duck
 Image Segmentation (Class Map)

What Is Image Segmentation?

Goal: Assign a class label to every pixel in an image.



■ Cat | ■ Dog | ■ Duck
 Image Segmentation (Class Map)



1 Cat | 3 Dog | 2 Duck | 0 Background
 Image Segmentation (Class Map)

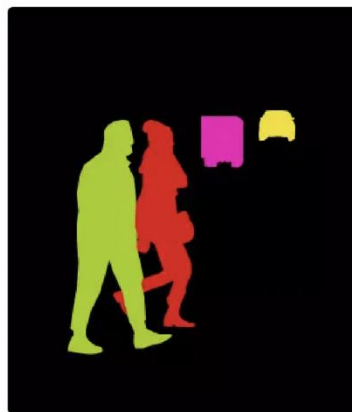
Output shape: same spatial dimensions as input ($H \times W$), with one class per pixel.

Three Types of Segmentation



**SEMANTIC IMAGE
SEGMENTATION**

Every pixel gets a class
— no instance distinction.



**INSTANCE
SEGMENTATION**

Each object instance gets
a unique mask.



**PANOPTIC
SEGMENTATION**

The complete picture —
most challenging.

Type	Classes?	Instances?	Background?	Example
Semantic	✓	✗	✓ Labeled	Road scene: all cars are one class
Instance	✓	✓	✗ Not labeled	Separate each cell in microscopy
Panoptic	✓	✓	✓ Labeled	Full scene understanding

Image: scaler.com

Why Is Segmentation Hard?

1. Scale

Tiny and huge objects in same image.
Need multi-scale feature extraction.

2. Boundaries

Where exactly does one object end
and another begin?

3. Ambiguity

Shadow or road? Reflections?
Difficult even for humans.

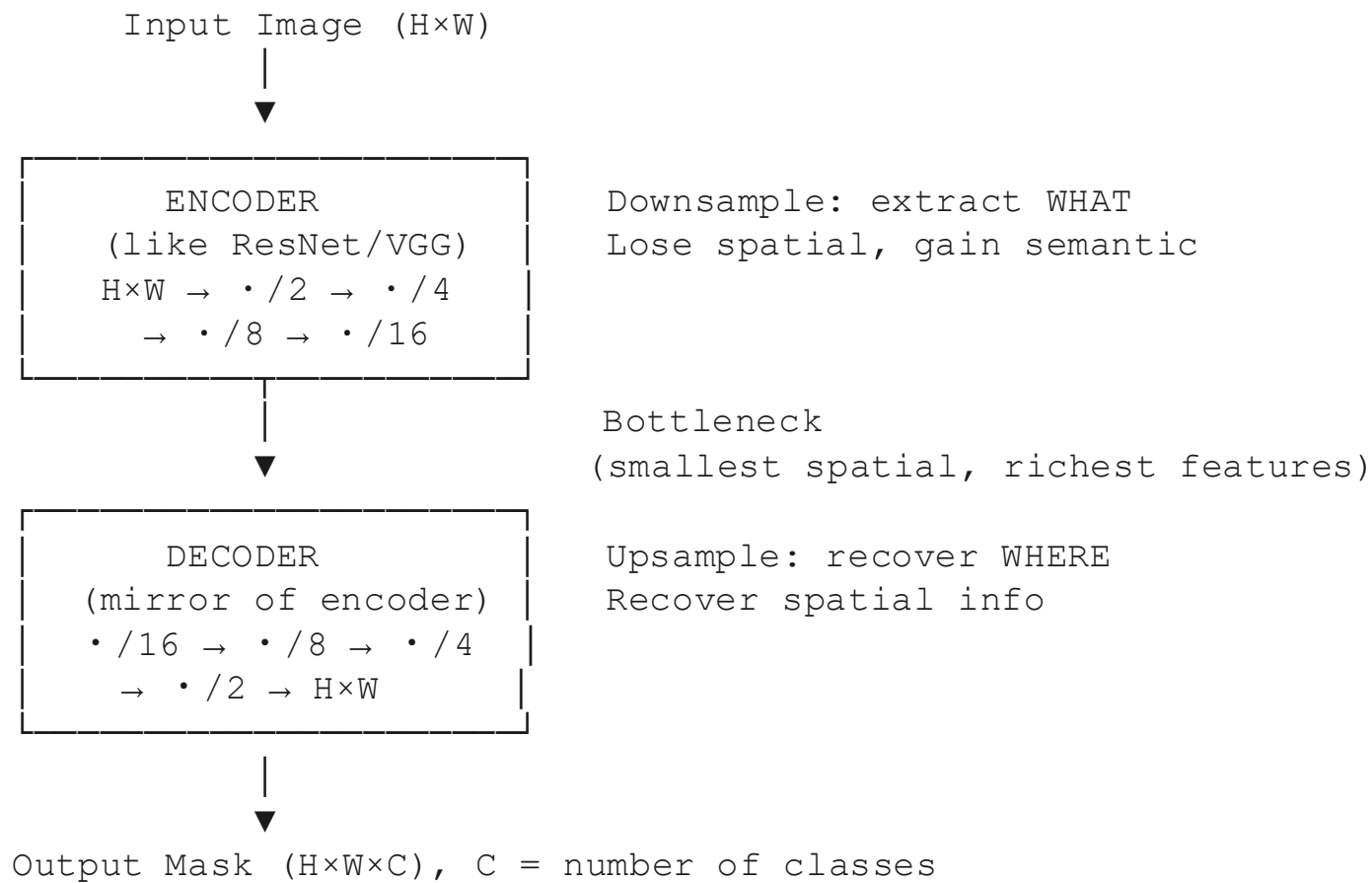
4. Class Imbalance

Tumor = 2% of pixels,
Background = 98%.

2. Encoder-Decoder Architectures

The Framework for Segmentation

The Core Idea: Encode, Then Decode



Why Not Just Use a CNN?

A regular CNN (for classification):

Image $\rightarrow$ [Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$...] $\rightarrow$ Global Pool $\rightarrow$ Class

Spatial resolution shrinks at each layer

448 $\rightarrow$ 224 $\rightarrow$ 112 $\rightarrow$ 56 $\rightarrow$ 28 $\rightarrow$ 14 $\rightarrow$ 7 $\rightarrow$ 1

$\uparrow$

Lost ALL spatial info!

For segmentation we need spatial info back!

Image $\rightarrow$ [Encoder] $\rightarrow$ [Decoder] $\rightarrow$ Pixel Labels

Shrink

448 $\rightarrow$ 7

(what is here?)

Grow back

7 $\rightarrow$ 448

(where is it?)

Upsampling Methods

How do we go from small feature maps back to full resolution?

1. Nearest Neighbor

Copy each value to neighbors.
Simplest, blocky results.

2. Bilinear Interpolation

Smooth interpolation between
values — less blocky.

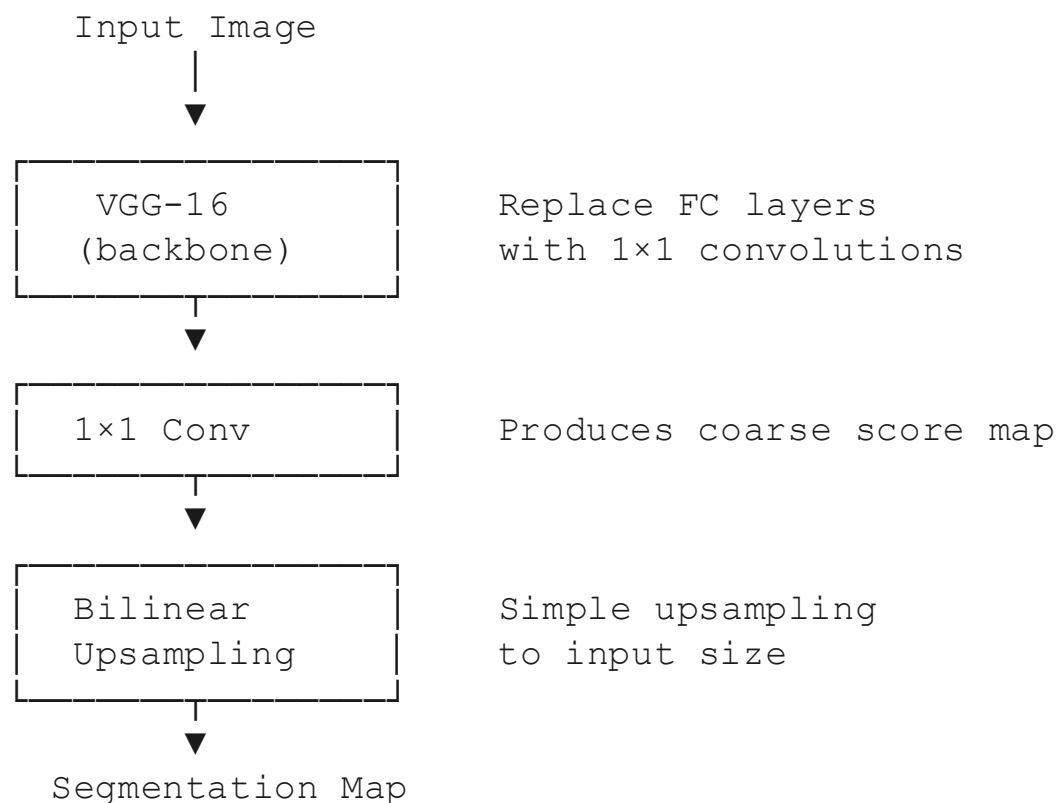
3. Transposed Convolution

Network LEARNS how to
upsample. Trained end-to-end.

U-Net uses transposed convolutions — the decoder learns what matters

Early Attempts: Fully Convolutional Network

Long et al. (2015) — The first deep segmentation model.

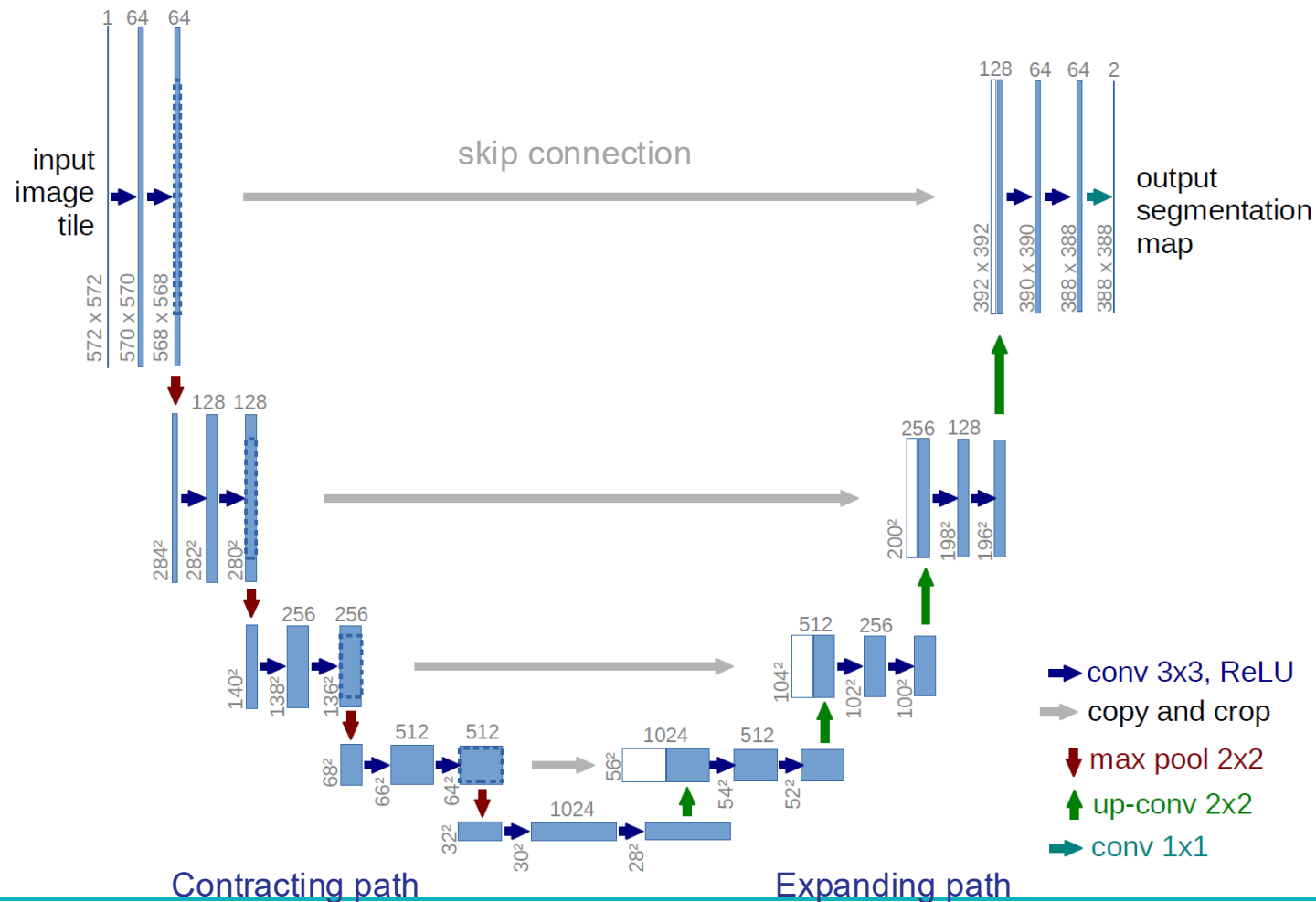


Limitation: Upsampling from very low resolution → blurry boundaries.

3. U-Net Architecture

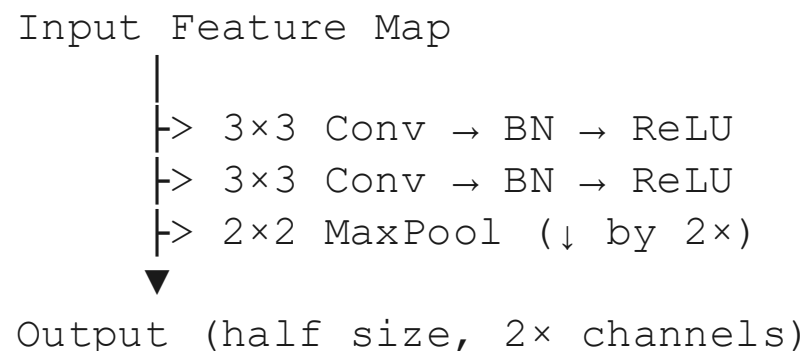
The Gold Standard for Medical Image Segmentation

U-Net: The Big Picture



U-Net: Encoder (Contracting Path)

Each encoder block:



Level	Spatial Size	Channels	What It Captures
1	568 × 568	64	Edges, textures
2	280 × 280	128	Simple patterns
3	136 × 136	256	Object parts
4	64 × 64	512	Object-level features
5 (Bottleneck)	28 × 28	1024	High-level semantics

U-Net: Decoder (Expanding Path)

Each decoder block:

Input Feature Map (from below)

- > 2×2 Transposed Conv ($\uparrow$ by 2×)
- > Concatenate with skip connection
(adds fine-grained spatial info!)
- > 3×3 Conv $\rightarrow$ BN $\rightarrow$ ReLU
- > 3×3 Conv $\rightarrow$ BN $\rightarrow$ ReLU

▼

Output (double size, half channels)

The decoder mirrors the encoder exactly — this symmetry gives U-Net its name

Skip Connections: The Secret Sauce

Without Skip Connections:

Encoder		Decoder
Level 1: fine details	> Lost!	Level 1: blurry output
Level 2: medium features	> Lost!	Level 2: approximate shapes
Level 3: semantic info	>>>>>	Level 3: knows WHAT, not WHERE

With Skip Connections:

Encoder		Decoder
Level 1: fine details	>>>>>	Level 1: precise boundaries!
Level 2: medium features	>>>>>	Level 2: accurate shapes!
Level 3: semantic info	>>>>>	Level 3: knows WHAT and WHERE

Skip connections transfer fine-grained spatial information from encoder to decoder.

Skip Connections vs ResNet Residuals

These are different — don't confuse them!

	U-Net Skip Connections	ResNet Residual Connections
Purpose	Transfer spatial info to decoder	Ease gradient flow
Where	Between encoder & decoder (horizontal)	Within same path (vertical)
Operation	Concatenation (double channels)	Addition (same channels)
What they carry	Fine-grained features from early layers	Identity mapping

U-Net:

Encoder == concat == Decoder

ResNet:

Block Input — add — Block Output
 └─ Conv ─┘

U-Net: Why It Works for Medical Images

Medical imaging has unique challenges that U-Net addresses:

Challenge	U-Net Solution
Limited data (few annotated images)	Skip connections preserve detail from few samples
Precise boundaries needed (tumor edges)	Multi-scale skip connections recover fine boundaries
Variable object sizes	Encoder captures multi-scale features
Strong prior on structure	Symmetric architecture encodes structural prior

U-Net achieves excellent results even with ~30 annotated images!

U-Net Implementation: DoubleConv Block

```
class DoubleConv(nn.Module):
    """Two 3x3 convolutions with BatchNorm and ReLU"""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        return self.conv(x)
```

This is the basic building block — used in both encoder and decoder.

U-Net Implementation: Full Model

```
class UNet(nn.Module):
    def __init__(self, in_channels=1, num_classes=2):
        super().__init__()
        # Encoder (contracting path)
        self.enc1 = DoubleConv(in_channels, 64)
        self.enc2 = DoubleConv(64, 128)
        self.enc3 = DoubleConv(128, 256)
        self.enc4 = DoubleConv(256, 512)
        self.bottleneck = DoubleConv(512, 1024)
        self.pool = nn.MaxPool2d(2)

        # Decoder (expanding path)
        self.up4 = nn.ConvTranspose2d(1024, 512, 2, stride=2)
        self.dec4 = DoubleConv(1024, 512) # 512+512 skip
        self.up3 = nn.ConvTranspose2d(512, 256, 2, stride=2)
        self.dec3 = DoubleConv(512, 256) # 256+256 skip
        self.up2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.dec2 = DoubleConv(256, 128) # 128+128 skip
        self.up1 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.dec1 = DoubleConv(128, 64) # 64+64 skip
        self.final = nn.Conv2d(64, num_classes, 1)
```

U-Net Implementation: Forward Pass

```
def forward(self, x):  
    # Encoder  
    e1 = self.enc1(x)                # → 64 ch  
    e2 = self.enc2(self.pool(e1))    # → 128 ch  
    e3 = self.enc3(self.pool(e2))    # → 256 ch  
    e4 = self.enc4(self.pool(e3))    # → 512 ch  
  
    # Bottleneck  
    b = self.bottleneck(self.pool(e4))  
                                     # → 1024 ch  
  
    # Decoder + skip connections  
    d4 = self.up4(b)  
    d4 = torch.cat([d4, e4], dim=1)  # Skip!  
    d4 = self.dec4(d4)               # → 512 ch
```

```
    d3 = self.up3(d4)  
    d3 = torch.cat([d3, e3], dim=1)  # Skip!  
    d3 = self.dec3(d3)               # → 256 ch  
  
    d2 = self.up2(d3)  
    d2 = torch.cat([d2, e2], dim=1)  # Skip!  
    d2 = self.dec2(d2)               # → 128 ch  
  
    d1 = self.up1(d2)  
    d1 = torch.cat([d1, e1], dim=1)  # Skip!  
    d1 = self.dec1(d1)               # → 64 ch  
  
    return self.final(d1)           # → num_classes ch
```

4. Loss Functions for Segmentation

Measuring Pixel-Level Quality

Why Standard Cross-Entropy Isn't Enough

Typical medical image



Background: ~97%

Tumor: ~3%

If model predicts ALL background:
 Cross-entropy loss is LOW (97% correct!)
 But completely useless for diagnosis!

We need losses that care about the small regions

Loss 1: Pixel-wise Cross-Entropy

The simplest segmentation loss — standard CE applied per pixel.

$$L_{CE} = -\frac{1}{N} \sum_{pixels} \sum_{c=1}^C y_c \cdot \log(p_c)$$

Pros:

- Simple and well-understood
- Works well when classes are balanced

Cons:

- Dominated by majority class (background)
- Model can "cheat" by predicting all background

Improvement: Weighted Cross-Entropy

Give higher weight to rare classes:

```
w_background = 1.0
```

```
w_tumor = 10.0 ← Force model to care about tumors!
```

Loss 2: Dice Loss

Based on the Dice coefficient — measures overlap between prediction and ground truth.

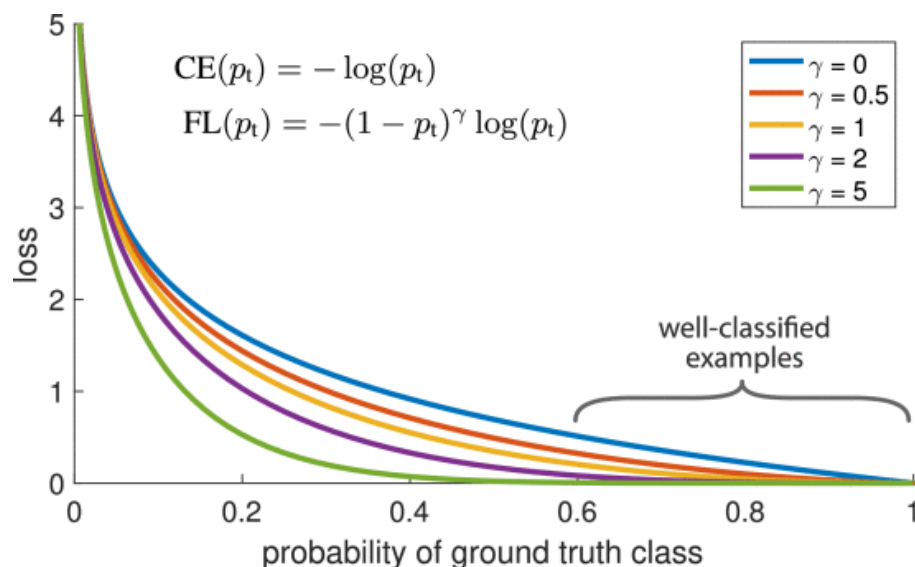
$$L_{DICE} = 1 - DICE = 1 - \frac{2 \times |\text{Prediction} \cap \text{Ground Truth}|}{|\text{Prediction}| + |\text{Ground Truth}|}$$

```
def dice_loss(pred, target, smooth=1e-6):  
    pred = torch.sigmoid(pred)  
    intersection = (pred * target).sum()  
    union = pred.sum() + target.sum()  
    dice = (2. * intersection + smooth) / (union + smooth)  
    return 1 - dice
```

Key advantage: Directly optimizes overlap — not dominated by background pixels!

Loss 3: Focal Loss

Lin et al. (2017) — Designed to handle extreme class imbalance.



$$p_t = \begin{cases} p, & \text{if } y = 1 \\ 1 - p, & \text{otherwise} \end{cases}$$

$$FL = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

p_t (confidence)	$(1 - p_t)^\gamma$	Effect
0.9 (easy, well-classified)	0.01	Loss reduced 100× → ignored!
0.5 (borderline)	0.25	Still meaningful loss
0.1 (hard, misclassified)	0.81	Near-full loss → model focuses here

Focal loss makes the model focus on HARD examples, ignoring the easy background pixels.

Focal Loss: Visual Intuition

Standard Cross-Entropy:

Easy pixels (background): 
 Hard pixels (tumor edge): 

← Many easy pixels: Dominates training!
 ← Few hard pixels: Barely contributes

Focal Loss:

Easy pixels (background): 
 Hard pixels (tumor edge): 

← Down-weighted!
 ← Model focuses here!

```
def focal_loss(pred, target, gamma=2.0, alpha=0.25):
    bce = F.binary_cross_entropy_with_logits(pred, target, reduction='none')
    p_t = torch.exp(-bce)
    alpha_t = alpha * target + (1 - alpha) * (1 - target)
    focal_weight = alpha_t * (1 - p_t) ** gamma
    return (focal_weight * bce).mean()
```

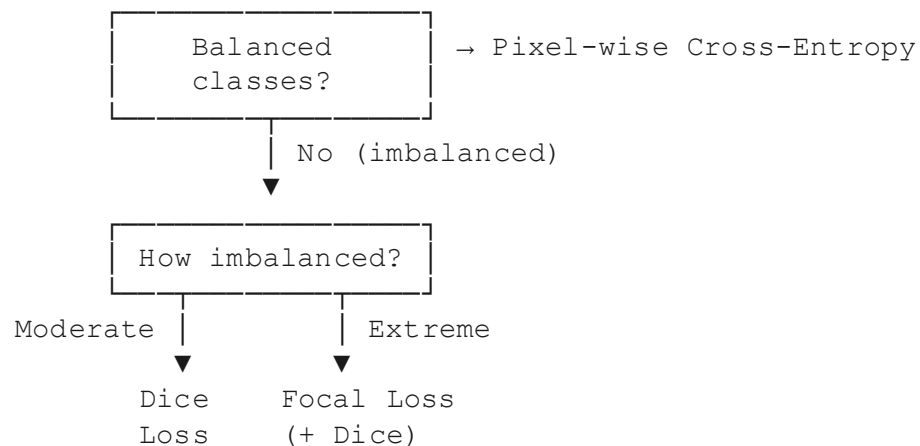
Combined Loss: Best of Both Worlds

```
class CombinedLoss(nn.Module):  
    def __init__(self, bce_weight=0.5, dice_weight=0.5):  
        super().__init__()  
        self.bce_weight = bce_weight  
        self.dice_weight = dice_weight  
  
    def forward(self, pred, target):  
        bce = F.binary_cross_entropy_with_logits(pred, target)  
        dice = dice_loss(pred, target)  
        return self.bce_weight * bce + self.dice_weight * dice
```

Loss	Strength	Weakness
BCE	Stable gradients, well-calibrated	Affected by class imbalance
Dice	Overlap-focused, handles imbalance	Unstable gradients for small objects
Focal	Focuses on hard examples	Requires tuning γ
BCE + Dice	Stable + overlap-aware	(Common default for medical segmentation)

Loss Functions: How to Choose

Choose your loss based on your problem:



**Safe Default:
BCE + Dice combined loss**

5. Evaluation Metrics

How Good Is Your Segmentation?

Dice Coefficient (F1 Score)

$$DICE = \frac{2 \times |\text{Prediction} \cap \text{Ground Truth}|}{|\text{Prediction}| + |\text{Ground Truth}|}$$

Range: [0, 1] 0 = no overlap 1 = perfect overlap

Example:

Ground Truth:

0	0	1	1	0
0	1	1	1	0
0	0	1	0	0

Prediction:

0	0	1	1	1
0	0	1	1	0
0	0	1	1	0

GT pixels = 6 Pred pixels = 7 Intersection = 5
 Dice = $(2 \times 5) / (6 + 7) = 10/13 = 0.769$

IoU (Jaccard Index)

Same IoU from object detection, but for pixel regions:

$$IoU = \frac{|\text{Prediction} \cap \text{Ground Truth}|}{|\text{Prediction} \cup \text{Ground Truth}|} = \frac{|\text{Intersection}|}{|\text{Prediction}| + |\text{Ground Truth}| - |\text{Intersection}|}$$

Same example: $IoU = 5 / (6 + 7 - 5) = 5/8 = 0.625$

Relationship: Dice is always $\geq$ IoU for the same prediction:

Dice	IoU	Quality
0.0	0.0	No overlap
0.5	0.33	Poor
0.7	0.54	Moderate
0.8	0.67	Good
0.9	0.82	Excellent
1.0	1.0	Perfect

Pixel Accuracy — Beware!

$$\text{Pixel Accuracy} = \frac{\text{Number of correctly classified pixels}}{\text{Total number of pixels}}$$

Why it can be misleading:

Image: 96% background, 4% tumor

Model predicts ALL BACKGROUND:

Pixel accuracy = 96% ← looks great!

Dice = 0.0 ← completely failed!

IoU = 0.0 ← completely failed!

Always report Dice and IoU alongside pixel accuracy

Evaluation Metrics Summary

Metric	Formula	Range	Handles Imbalance?
Pixel Accuracy	Correct / Total	[0, 1]	✗ No
Dice (F1)	$2 \times TP / (2 \times TP + FP + FN)$	[0, 1]	✓ Yes
IoU (Jaccard)	$TP / (TP + FP + FN)$	[0, 1]	✓ Yes
Mean Dice	Average Dice across classes	[0, 1]	✓ Yes
Mean IoU (mIoU)	Average IoU across classes	[0, 1]	✓ Yes

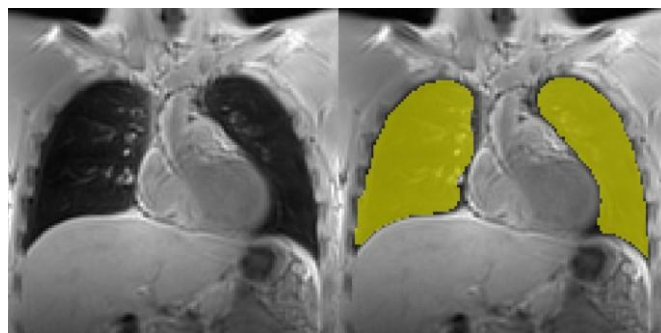
Standard reporting:

- Medical: Dice coefficient (per class + mean)
- Autonomous driving, general: mIoU
- Competitions (e.g., COCO): IoU-based metrics

6. Applications of Segmentation

Medical Imaging

Chest X-ray



DOI: 10.1186/s12880-021-00608-1

Brain MRI



DOI: 10.1038/s41598-024-71250-4

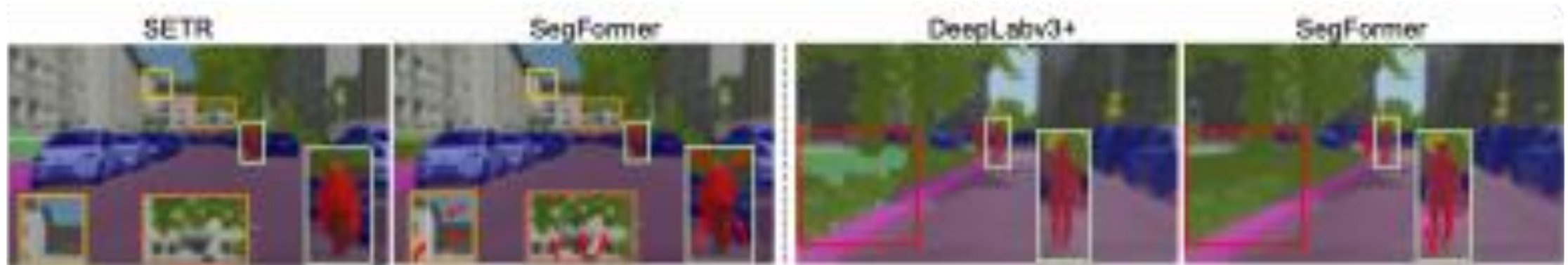
Cell microscopy



DOI: 10.1007/978-3-319-24574-4_28

Application	Dataset	Typical Dice
Lung segmentation	Montgomery/JSRT	~0.96
Brain tumor	BraTS	~0.85
Cell nuclei	Data Science Bowl 2018	~0.90
Retinal vessels	DRIVE	~0.80

Autonomous Driving



<https://dl.acm.org/doi/10.5555/3540261.3541185>

Cityscapes dataset: 30 classes, every pixel annotated (road, car, pedestrian, building, sky...)

Top models: DeepLabV3+, SegFormer → mIoU > 80%

Other Applications

Domain	Task	Impact
Satellite Imagery	Land use classification	Urban planning, agriculture monitoring
Document Analysis	Layout detection	OCR preprocessing, form understanding
Fashion	Clothing segmentation	Virtual try-on, e-commerce
Agriculture	Crop/weed segmentation	Precision farming, herbicide reduction
Robotics	Scene understanding	Grasping, navigation

Summary: Key Concepts

Concept	Key Idea
Semantic Segmentation	Classify every pixel (no instance distinction)
Instance Segmentation	Separate each object instance with a mask
Encoder-Decoder	Downsample for semantics, upsample for spatial detail
U-Net	Symmetric encoder-decoder with skip connections
Skip Connections	Transfer fine-grained spatial info from encoder to decoder
Dice Loss	Overlap-based loss, handles class imbalance
Focal Loss	Down-weights easy examples, focuses on hard cases
Dice Coefficient	Standard segmentation metric (range 0-1)
IoU / Jaccard	Intersection over union for pixel regions

Common Issues & Solutions

Issue	Solution
Model predicts all background	Use Dice loss or weighted BCE; check mask loading
Masks look blocky/blurry	Resize masks with NEAREST; check resolution match
Loss is NaN	Reduce learning rate; add smooth term to Dice loss
Very different train vs val metrics	Overfitting — add augmentation, try dropout
Out of GPU memory	Reduce batch size, image size, or use smaller model
Dice score stuck at ~0.5	Check mask values are 0/1; check normalization
Masks not aligning with images	Check file sorting; visualize before training!



15 minutes

Hands-On Lab

Medical Image Segmentation (90 minutes)

Common Issues & Solutions

Issue	Solution
Model predicts all background	Use Dice loss or weighted BCE; check mask loading
Masks look blocky/blurry	Resize masks with NEAREST; check resolution match
Loss is NaN	Reduce learning rate; add smooth term to Dice loss
Very different train vs val metrics	Overfitting — add augmentation, try dropout
Out of GPU memory	Reduce batch size, image size, or use smaller model
Dice score stuck at ~0.5	Check mask values are 0/1; check normalization
Masks not aligning with images	Check file sorting; visualize before training!

Tips for Success

Do

- Visualize data BEFORE training
- Start with small model and small images
- Plot training curves frequently
- Use combined loss (BCE + Dice)
- Apply data augmentation
- Check predictions every few epochs

Don't

- Skip data visualization
 - Train on full resolution without checking
 - Use only pixel accuracy as metric
 - Forget to apply SAME augmentation to image AND mask
 - Train too many epochs without validation
-

Bonus Challenges

If you finish early:

- 1. Multi-Class Segmentation** Extend to 3+ classes (e.g., heart, lungs, background) — modify output channels and loss
 - 2. Architecture Comparison** Compare U-Net vs DeepLabV3+ vs FPN using `segmentation_models_pytorch`
 - 3. Transfer Learning Impact** Compare random initialization vs ImageNet pre-trained encoder
 - 4. Augmentation Ablation** Train with no augmentation vs basic vs heavy — report Dice for each
 - 5. Boundary Refinement** Implement CRF post-processing or boundary-aware loss
-

What We Covered Today

1. **Segmentation taxonomy** — semantic, instance, panoptic
 2. **Encoder-decoder** — downsample for semantics, upsample for spatial
 3. **U-Net** — symmetric architecture with skip connections
 4. **Skip connections** — transfer fine-grained spatial info across the U
 5. **Loss functions** — Dice loss handles class imbalance, combine with BCE
 6. **Evaluation** — Dice coefficient and IoU, not just pixel accuracy
-

Looking Ahead

Next Week: Explainable AI (XAI) for Computer Vision

- Why model explainability matters
- Saliency maps: gradient-based visualization
- GradCAM: understanding what your model "sees"
- Applying XAI to your segmentation and classification models
- When to trust (and distrust) explanations

Preparation:

- Reading: GradCAM paper (Selvaraju et al., 2017)
 - Think about: "When have you needed to explain a model's decision?"
-

Questions?

Q: Can I use U-Net for non-medical images?

A: Absolutely! U-Net works for any segmentation task — satellite imagery, document analysis, etc.

Q: What about 3D segmentation (CT/MRI volumes)?

A: 3D U-Net exists! Replace 2D convolutions with 3D — same architecture but computationally heavier.

Q: How many training images do I need?

A: U-Net was designed for limited data. 30-100 images can work with proper augmentation.

Q: Dice loss or Focal loss?

A: Start with Dice + BCE (safe default). Switch to Focal if you have extreme imbalance with tiny foreground.

Thank You!

See you next week!

Lab submission due in 1 week
